

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

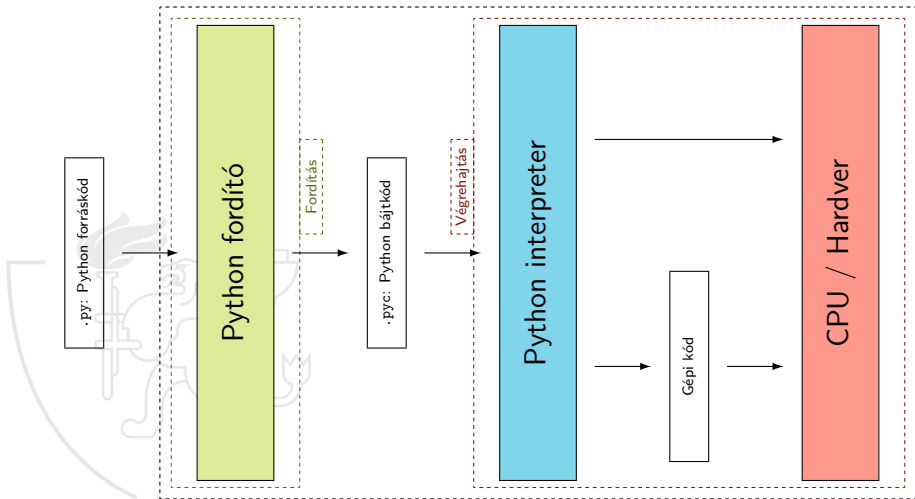
Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A Python program végrehajtásának folyamata

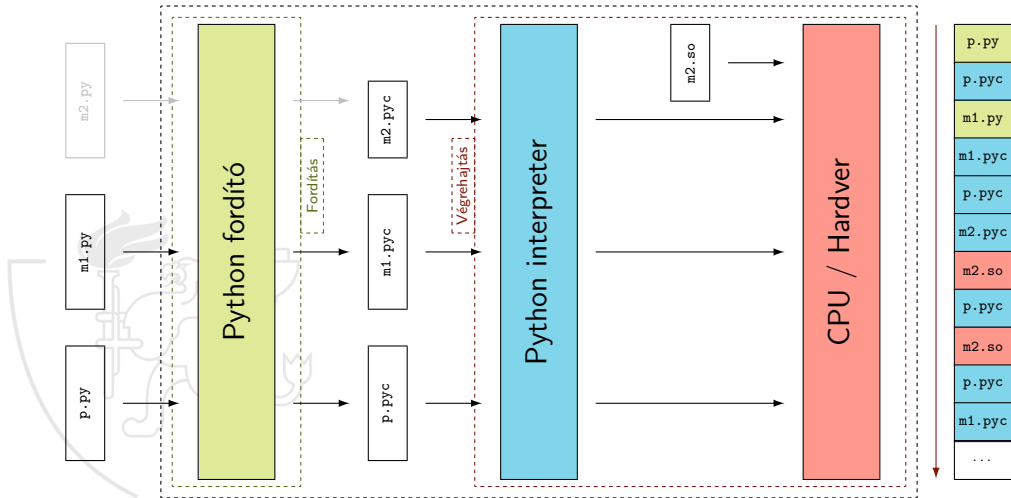
- A Python programok futtatása egyszerű
 - A Python programunkat tartalmazó .py kiterjesztésű fájlt odaadjuk a Python „rendszernek” (python, esetleg python3), ami értelmezi és futtatja azt.
- A Python programok futtatása komplex
 - A fordító lefordítja a Python forráskódot egy Python *bájtkód*-ra, ha közben *szintaktikai hibát* talál, azt jelzi, és megáll a folyamat.
 - A lefordított bájkódot az interpreter (Python Virtuális Gép – PVM) értelmezi és végrehajtja. *Futási hiba* esetén elindul a megfelelő hibakezelési folyamat.
 - Előre megírt Python program futtatásakor a két lépés a teljes programra egymás után következik be.
 - Interaktív használat esetén a fordítás-végrehajtás lépései utasításonként felváltva ismétlődnek.
 - Egyes Python környezetek egyéb technológiákat is bevethetnek (pl. Just-In-Time fordítás).

A Python program végrehajtásának folyamata



- Egy `import` parancs esetén a megadott modul betöltődik és lefut, majd a vezérlés visszaadódik az importáló programnak.
- Egy modul (egy végrehajtás során) egyszer importálódik (a legelső rá vonatkozó `import` utasítás végrehajtásakor), minden további rá vonatkozó `import` utasítás hatástalan lesz.
- Modulok importálása során előfordulhat, hogy nem a modul forráskódja, hanem rögtön a Python bájtkód (.pyc) lesz felhasználva (így a fordítási lépés kimaradhat, ha nincs rá szükség).
- A modulok más nyelven (például C/C++ nyelven) megírt, gépi kódra fordított kódrészleteket is tartalmazhatnak.

A Python program végrehajtásának folyamata



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok**
 - Python unpacking
 - Kivételkezelés

Függvényparaméterek, függvényargumentumok

- Amikor az eljárásvezérlést néztük, akkor azt állítottuk, hogy minden függvénynek adott számú és (paraméterenként akár különböző) típusú paramétere van, és az adott függvény meghívásánál pontosan ennyi számú és minden paraméterhez a megfelelő típusú argumentumot kell megadni.
- Azután láttuk, hogy a Python „lazán” kezeli a típusokat, tehát az „azonos típusú” kitétel Python-ban nem él (vagy legalábbis nem fordítási időben, hanem futásidőben van ellenőrizve és kezelve).
- Aki nem vette volna észre: Pythonban a fix darabszám sem feltétlenül szigorú kitétel, és néha konkrétan azt is megadtuk, hogy sorrendtől függetlenül melyik paraméter értékét adjuk meg éppen.
- Nézzük meg, Pythonban milyen lehetőségeink vannak az argumentumok megadására és a paraméterek definiálására.

- Python-ban kétféleképpen tudunk függvényargumentumokat megadni egy függvény hívásakor
 - **Sorrendi (*positional*) argumentumok:** Amikor a függvényhívás zárójelein belül pusztán felsorolunk kifejezéseket, azok értékei a megadott sorrendben kerülnek átadásra a függvény paraméterei számára.
 - **Kulcsszavas (*keyword*) argumentumok:** Amikor a függvényhívás zárójelein belül kulcsszóval és értékadással adunk meg kifejezéseket, azok a kulcsszó alapján adódnak át a függvény megfelelő nevű paraméterei számára.
- A két argumentummegadási mód együtt is használható, de a sorrendi argumentumok meg kell, hogy előzzék a kulcsszavas argumentumokat!

Függvényargumentumok

Sorrendi és kulcsszavas argumentumok

```
>>> def function(a, b, c):  
...     print(f"{a=!r}␣{b=!r}␣{c=!r}")  
...  
>>> function(1,2,3)  
a=1 b=2 c=3  
>>> function(1,c=2,b=3)  
a=1 b=3 c=2  
>>> function(1,c=2,a=3)  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: function() got multiple values for argument 'a'  
>>> function(c=1,2,3)  
  File "<stdin>", line 1  
    function(c=1,2,3)  
                ^  
SyntaxError: positional argument follows keyword argument
```

- Az alapvetően igaz, hogy ha egy függvénynek van valahány paramétere, az azért van, mert ezeket fel szeretné használni.
- Akkor mégis hogyan lehetséges, hogy ugyanazt a függvényt több és kevesebb argumentummal is meg tudjuk hívni?
- Háromféleképpen:
 - **Alapértelmezett (*default*) értékekkel:** Az egyes paramétereknek adhatunk egy default értéket. Ha az adott paraméterhez tartozó argumentum hiányzik a függvényhíváskor, akkor a függvény ezt a default értéket fogja használni. (A default érték egyszer, a függvény *definíciója* közben kerül meghatározásra!)
 - **Változó hosszú sorrendi (*arbitrary positional*) argumentumlistával:** Ha a függvényünk szándékosan (további) akármennyi sorrendi argumentumot fogadhat, ezeket egyetlen speciális, `*` karakterrel jelzett paraméteren keresztül `tuple`-ként kaphatjuk meg.
 - **Változó hosszú kulcsszavas (*arbitrary keyword*) argumentumlistával:** Ha a függvényünk szándékosan (további) akármennyi kulcsszavas argumentumot fogadhat, ezeket egyetlen speciális, `**` karakterekkel jelzett paraméteren keresztül `dict`-ként kaphatjuk meg.

Függvényparaméterek

Alapértelmezett értékek és változó hosszú sorrendi argumentumlista

```
>>> def f(a, b='b', *c):  
...     print(f"{a=!r}_ {b=!r}_ {c=!r}")  
...  
>>> f()
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

TypeError: f() missing 1 required positional argument: 'a'

```
>>> f(1)  
a=1 b='b' c=()  
>>> f(1,2)  
a=1 b=2 c=()  
>>> f(1,2,3)  
a=1 b=2 c=(3,)  
>>> f(1,2,3,4,5)  
a=1 b=2 c=(3, 4, 5)
```

Függvényparaméterek

Alapértelmezett érték kiértékelése

```
>>> def f(list_=[]):  
...     list_.append(len(list_))  
...     return list_  
...  
>>> f([])  
[0]  
>>> f()  
[0]  
>>> f(['a', 'b'])  
['a', 'b', 2]  
>>> f()  
[0, 1]  
>>> f()  
[0, 1, 2]  
>>> f([])  
[0]
```

Függvényparaméterek

Változó hosszú kulcsszavas argumentumlista

```
>>> def f(p, *a, **k):  
...     print(f"{p=!r}_ {a=!r}_ {k=!r}")  
...  
>>> f(1,2,3,4)  
p=1 a=(2, 3, 4) k={}  
>>> f(1,2,3,4,x=5,y=6)  
p=1 a=(2, 3, 4) k={'x': 5, 'y': 6}  
>>> f(1,x=2,y=3)  
p=1 a=() k={'x': 2, 'y': 3}  
>>> f(x=1,y=2,p=3)  
p=3 a=() k={'x': 1, 'y': 2}
```

Függvényparaméterek

Sorrendi és kulcsszavas mód kikényszerítése

- Általános esetben bármely nevesített (nem változó hosszú) paraméter megadható sorrendi vagy kulcsszavas argumentummal is (persze figyelve a korábban említett „először sorrendi, azután kulcsszavas” sorrendre).
- Kikényszeríthető-e, hogy valamely paraméter csak sorrendi, vagy csak kulcsszavas módon legyen megadható?
- Igen!
 - Ha a paraméterlistában szerepel egy `*args` alakú változó hosszú sorrendi argumentumot kezelő paraméter, akkor az utána következő paramétereket csak kulcsszavas argumentumként tudjuk megadni, mert az `*args` mohó, és a további sorrendi argumentumokat „lenyeli”.
 - A csak kulcsszavas viselkedés kikényszeríthető változó hosszú sorrendi argumentumot kezelő paraméter nélkül is; ha a paraméterlistában szerepel egy `*` (önmagában), annak ugyanilyen hatása van a további paraméterekre.
 - Hasonlóan, ha a paraméterlistában szerepel egy `/` (önmagában), az előtte lévő paraméterek csak és kizárólag sorrendi argumentummal adhatók meg.

Függvényparaméterek

Sorrendi és kulcsszavas mód kikényszerítése

```
>>> def f(positional, /, both, *, keyword):  
...     print(f"{positional=!r}_ {both=!r}_ {keyword=!r}")  
...  
>>> f(1, 2, 3)
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

TypeError: f() takes 2 positional arguments but 3 were given

```
>>> f(keyword=1, both=2, positional=3)
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

TypeError: f() got some positional-only arguments passed as keyword arguments: 'positional'

```
>>> f(1, 2, keyword=3)
```

positional=1 both=2 keyword=3

```
>>> f(1, keyword=2, both=3)
```

positional=1 both=3 keyword=2

Függvényargumentumok

Sorrendi és kulcsszavas argumentumlista megadása változóból

- A fenti módszerekkel egy adott függvényhívás kifejezésben még mindig a statikusan, fordítási időben megadott fix számú argumentumot tudjuk átadni.
- Van-e mód arra, hogy az átadott argumentumokat is dinamikusan, futási időben állítsuk össze?
- Igen!
 - Sorrendi paramétereket meg tudunk adni a `*` és (dinamikusan összeállított) `list` vagy `tuple` típus segítségével. Az argumentumlistában található `*list_` argumentum esetén a `list_` értékei sorrendi argumentumként be lesznek helyettesítve az argumentumlistába a `list_` argumentum helyére.
 - Kulcsszavas paramétereket meg tudunk adni a `**` és (dinamikusan összeállított) `dict` típus segítségével. Az argumentumlistában található `**dict_` argumentum esetén a `dict_` kulcs-érték párojai kulcsszavas argumentumként be lesznek helyettesítve az argumentumlistába.

Függvényargumentumok

Sorrendi és kulcsszavas argumentumlista megadása változóból

```
>>> def f(a, *args, kw, **kwargs):
...     print(f"{a=!r}␣{args=!r}␣{kw=!r}␣{kwargs=!r}")
...
>>> pos1=['i', 'ii']
>>> pos2=('III', 'IV')
>>> kwr={ 'kw':1, 'ex':2}
>>> more={ 'ne':3.0, 'mo':4.0}
>>> f(*pos1,*pos2,**kwr,**more)
a='i' args=('ii', 'III', 'IV') kw=1 kwargs='ex': 2, 'ne': 3.0, 'mo': 4.0
>>> f('##',*pos1,**more,kw=0)
a='##' args=('i', 'ii') kw=0 kwargs='ne': 3.0, 'mo': 4.0
>>> f(*pos2,'@',**kwr,more='3.1415')
a='III' args=('IV', '@') kw=1 kwargs='ex': 2, 'more': '3.1415'
```

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking**
 - Kivételkezelés

Az *unpacking* operátor

- Olyat már láttunk, hogy egy listát több változónak adunk értékül.
 - Ilyen esetben a ha lista elemszáma és a változók száma nem egyezett, hibát kaptunk.
- Olyat is láttunk, hogy egy függvény argumentumait egy listából adjuk meg a `*` operátor segítségével.
 - Ez úgy működött, mintha a lista elemeit egyesével, adott sorrendben adtuk volna meg argumentumként.
- Tudtunk olyan függvényparamétert csinálni a `*` operátor segítségével, ami több argumentumot gyűjtött össze.
 - Ilyenkor az adott paraméter egy listát adott vissza, a megfelelő argumentumokkal feltöltve.
- A `*` operátor bizonyos megkötésekkel **többszörös** értékadásokban is működik.

list „becsomagolása”

- Többszörös értékadás esetén ha valamely célváltozó elé *-ot írunk, akkor az listaként össze fogja gyűjteni azokat az értékeket, amelyeknek „nem jut változó”.

```
>>> a = (1,2,3,4,5)
>>> b = [9,8,7,6,5]
>>> x, *y = a
>>> x
1
>>> y
[2, 3, 4, 5]
>>> (*u, v) = b
>>> u
[9, 8, 7, 6]
>>> v
5
```

```
>>> [k, *l, m] = a
>>> k
1
>>> l
[2, 3, 4]
>>> m
5
>>> m, n, *o, p, q, r = b
>>> o
[]
>>> p
7
```

list „becsomagolása”

- Akár üres lista is lehet az eredmény, de ha kevesebb az érték, mint a változó, az hiba!
- A `*` operátor egyszeres értékadásban nem használható!

```
>>> k = 1
>>> l = [2, 3, 4]
>>> m, n, *o, p, q, r = l
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: not enough values to unpack (expected at least 5, got 3)
>>> *x = k, m
File "<stdin>", line 1
SyntaxError: starred assignment target must be in a list or tuple
```

list és tuple „kicsomagolása”

- Többszörös értékadás esetén ha a jobb oldalon egyetlen lista áll, a célváltozók annak elemeit fogják megkapni.
- Többszörös értékadás esetén ha valamely listaérték elé `*`-ot írunk, akkor a lista elemei egyesével behelyettesítődnek a lista helyére.

```
>>> a, b = (1,2), [9,8]
>>> a
(1, 2)
>>> x, y = a
>>> x
1
>>> y
2
>>> p, q, r = a, *b
>>> p
(1, 2)
```

```
>>> q
9
>>> r
8
>>> k, l, m, n, o = *a, 5, *b
>>> k
1
>>> m
5
>>> n
9
```

list és tuple „kicsomagolása”

- A `*` operátor egyetlen értékre nem használható!
- a „ki-” és „becsomagolás” kombinálható.

```
>>> a, b = (1,2), [9,8]
>>> x = *a
File "<stdin>", line 1
SyntaxError: can't use starred expression here
>>> v, *w, z = *a, *b
>>> v
1
>>> w
[2, 9]
>>> z
8
```

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólagos számítás
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

- Alapfilozófia: *Rosszul formált kód nem fog jól futni.*
- Ideális lenne, ha minden hibát fordítási időben ki lehetne deríteni, de ez nem lehetséges.
- Általános megoldás lehet hibatűrő algoritmusok megalkotása és implementálása, de a gyakorlatban ez sem megvalósítható.
 - Ki szeretne olyan programot írni, amelyben 20 sor az érdemi rész, és még 200 csak azért kell, hogy az esetleges hibákat lekezeljük?
 - Ráadásul: utólag nehéz lehet megmondani, hogy a 220 sorból mi a lényeg, és mi kell pusztán a hibakezelés miatt.
- A megoldás: nyelvi szinten adjunk lehetőséget hibakezelésre, ami szintaktikailag elkülönül a „normál” kódtól.

- *Kivételes feltétel*: egy probléma, ami meggátolja a program futást egy adott függvényben (blokkban).
- Bizonyos kivételek (pl. `ZeroDivisionError`) automatikusan keletkezhetnek, de mi is tudunk kivételeket, hibákat dobni (a `raise` utasítás segítségével).
- Kivétel esetén a program normál futása megszakad, és egy kivételkezelő mechanizmus veszi át a vezérlést.
 - Ez kicsit olyan, mintha a függvény a hibával térne vissza oda, ahol meghívták, ez viszont a hívó függvényből nézve olyan, mintha a hívott függvény meghívásának helyén keletkezett volna a hiba.
 - Ha nem kezeljük a hibát, akkor ez a mechanizmus gyakorlatilag a program végét jelenti.
- *Védett régió*: a kód olyan része, ami kivételeket hozhat létre és amit hibakezelő kód követ.
- A kivételeket, hibákat a program adott pontjain „le tudjuk kezelni”, ilyen esetben a program visszatér „normál működési módba”.

A try-except utasítás

- Ha valamilyen lehetséges hiba következményeit szeretnénk kezelni.
 - Mit csináljunk, ha futási hiba lép fel egy utasítás végrehajtása során?
 - Ha a felhasználó nem számot ad meg, akkor nem számolunk.
 - De ha nullát, akkor is baj van.

```
try:
    v = int(input('Kérek egy egész számot: '))
    if v % 2 == 0:
        print('A szám páros, a reciproka %f!' % (1 / v))
    else:
        print('A szám páratlan!')
except:
    print('Valami hiba történt!')
```

A try-except utasítás

- Az utasításnak négyféle része van:

```
try:
    # Kód, amiben hiba keletkezhet
except Exception:
    # Mit csináljunk egy Exception esetén?
    # Több ilyen except blokk is lehet
else:
    # Mit csináljunk, ha nem volt hiba?
finally:
    # Ez mindenképpen lefut, mielőtt továbblépünk
```

Kivételkezelés [1/2]

exception.py [1–26]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-12-07
6cc-by-nc-nd
7
8Kivételkezelés
9"""
10import sys
11
12def f(mode):
13    if mode not in range(2):
14        raise ValueError("Az f()-ből dobjuk, mert rossz értéket kapott.")
15    if mode:
16        raise NameError("Az f()-ből dobjuk, mert nem 0-át kapott.")
17    print("[f] Ezt soha nem szabadna látnunk.")
18    print("[f] Nem dobunk hibát f()-ből.")
19
20def g(mode):
21    try:
22        if not mode.isnumeric():
23            raise TypeError("A g()-ből dobjuk, mert nem szám.")
24        print("[g] Ezt soha nem szabadna látnunk.")
25        f(int(mode))
26    print("[g] try: túlléptünk a függvényhíváson")
```

Kivételkezelés [2/2]

exception.py [27–49]

```
27 except ValueError:
28     print("[g]_except:_ValueError,_de_továbbdobjuk")
29     raise
30 else:
31     print("[g]_else:_ha_nem_volt_hiba")
32 finally:
33     print("[g]_finally:_mindenképpen_lefut")
34 print("[g]_Nem_dobunk_hibát_g()-ből.")
35
36 def main():
37     try:
38         g(sys.argv[1])
39         print("[main]_try:_túlléptünk_a_függvényhíváson")
40     except Exception as e:
41         print(f"[main]_except:\n\t{e}")
42     else:
43         print("[main]_else:_ha_nem_volt_hiba")
44     finally:
45         print("[main]_finally:_mindenképpen_lefut")
46     print("[main]_vége")
47
48 if __name__ == '__main__':
49     main()
```